Lecture 04
Turing Machines and Computability

Algorithms, AI & Data Science II

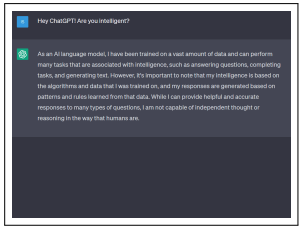
Prof. Dr. Ingo Scholtes
Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

- we introduced formal grammars and languages and studied the **Chomsky hierarchy**
- we introduced **automata models** that can **recognize languages**
- **which computational model is neccessary** to recognize language depends on its type in Chomsky hierarchy
- so far, we considered automata for **context-free (type 2)** and **regular (type 3)** languages

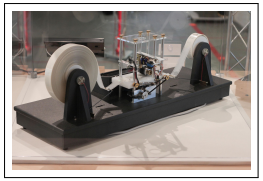
open questions

1. what are the limits of these computational models?
2. are there fundamental limits to what a machine can "compute"?



response from large language model ChatGPT

image credit: OpenAI screenshot, taken on 12/04/2023



physical model of a **Turing machine**

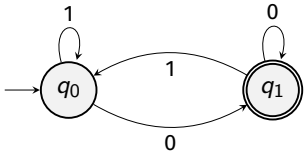
image credit: Rocky Acosta, Wikimedia Commons, CC-BY-SA

Notes:

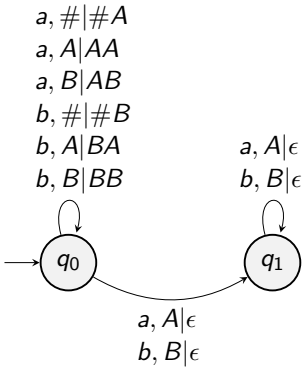
- **L04: Turing Machines and Computability** 06.05.2025
- **Educational objective:** We introduce Turing machines, explore the question which problems are fundamentally computable, and discuss Turing's definition of machine intelligence.
  - Turing machines
  - Limits of Decidability
  - Computability and Machine Intelligence
- **Handout version:** *2025/05/05 19:57:39*

Notes:

Automata models so far ...



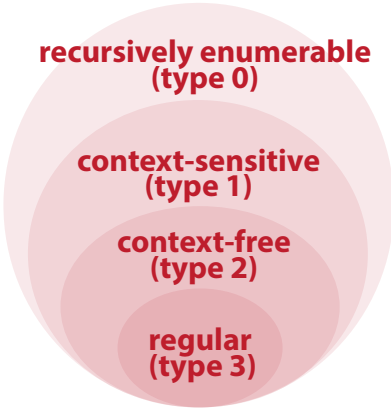
deterministic and non-deterministic finite automata (DFA/NFA)



deterministic and non-deterministic pushdown automata (NPDA/PDA)

Associated languages

- **regular languages** can be recognized by **finite automa**
- non-deterministic and deterministic finite automata are **equally powerful**, i.e. both recognize regular languages
- **context-free languages** can be recognized by **non-deterministic pushdown automata**
- **deterministic PDAs** recognize a **subset of context-free languages**
- what about type 0 and 1 languages?



Notes:

Notes:

Group Exercise 04-01

1. Ask ChatGPT whether the language $L = \{a^n b^n c^n | n \geq 1\}$ is context-free. Check whether the answer is correct.

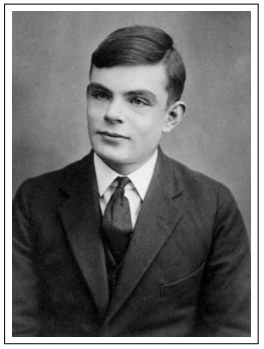
   ChatGPT wrongly states that L is context-free, arguing that for the following context-free grammar G we have $L(G) = L$: $G = (\{A, B\}, \{a, b\}, P, S)$ with

   $$P = \{S \to ABC$$
   $$A \to aA \mid \epsilon,$$
   $$B \to bB \mid \epsilon ,$$
   $$C \to cC \mid \epsilon \}$$

   However, we e.g. have $a \in L(G)$ and thus $L(G) \neq L$.

Beyond context-free languages ...

- consider language $L = \{a^n b^n c^n | n \geq 1\}$
- we can use so-called **pumping lemma** to prove that L is **not context-free** → exercise sheet
- implies that there exists **no non-deterministic or deterministic PDA** that recognizes L
- last-in/first-out (LIFO) access to stack and single "pass" over input tape limits expressiveness of PDAs
- idea: replace stack by (initially empty) **infinite tape**, that can be accessed by movable **read-write head**
- **Turing machines** proposed by Alan M Turing to formalize intuitive notion of **computability**



Alan M Turing (1912 – 1954)

image credit: Wikimedia Commons, public domain

Notes:

Notes:

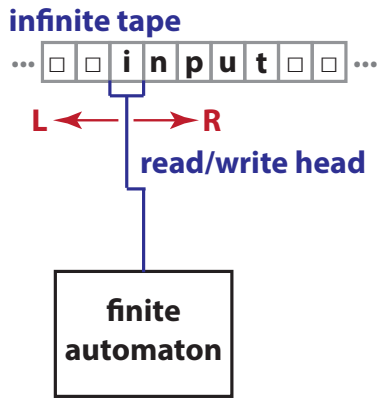
Turing machines

definition

A **Turing machine** (TM) is a seven-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, E)$ with

- Q finite set of states with **initial state** $q_0 \in Q$ and **final states** $E \subseteq Q$
- Σ finite alphabet of **input symbols**
- $\Gamma \supseteq \Sigma \cup \{\square\}$ finite alphabet of **tape symbols** where $\square$ is the **blank symbol**
- $\delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{L, R\}$ is a **transition relation**

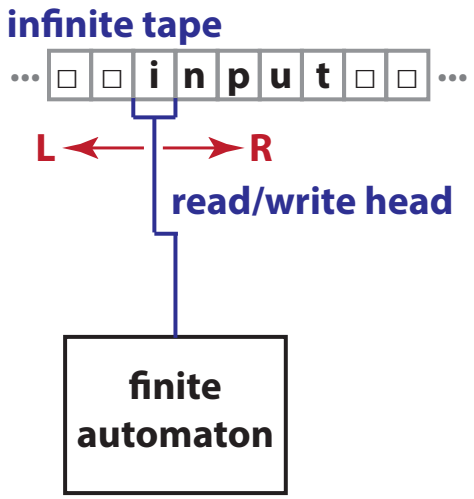
- we initially write input to tape, all other tape positions contain **blank symbol** $\square$
- in each step, Turing machine **reads current symbol under read/write head** and overwrites it by other symbol
- L and R indicate that **read/write head is moved left or right**, respectively
- **Turing machine is deterministic** (DTM) iff δ is functional, i.e. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ (NTM otherwise)



Turing machine

Turing machines and languages

- we use **graph to represent transitions**, where (q, x, q', x', H) is represented by edge $(q, q') \in E$ with label $x|x', H$
- we call $(\alpha, q, \beta) \in \Gamma^* \times Q \times \Gamma^*$ **configuration of** M, where
  - q is current state of M,
  - $\alpha\beta$ is non-empty part of the tape, and
  - read-write head is at first symbol of β
- δ defines **transitions between configurations**, i.e. $c_0 \to c_1 \to \ldots \to c_n$
- M accepts w iff it **halts in final state** q_e, i.e. starting from (ϵ, q_0, w) it reaches (α, q_e, β) after a **finite number of steps**
- M that never halts on w does not accept w

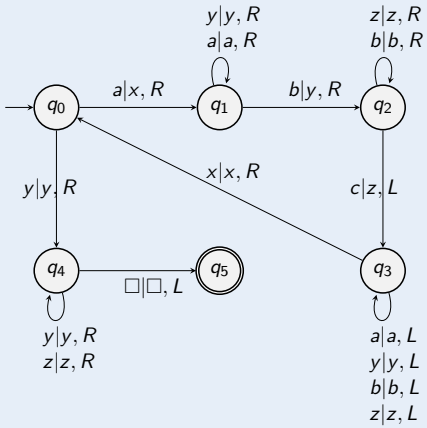


Notes:

Notes:

Group Exercise 04-02

1. Construct a Turing machine that recognizes the language $L = \{a^n b^n c^n | n \geq 1\}$.



Group Exercise 04-03

1. For the Turing machine from the previous exercise, compute the sequence of all configurations for the input word *aabbcc*.

$(\epsilon, q_0, aabbcc) \to (x, q_1, abbcc) \to (xa, q_1, bbcc) \to (xay, q_2, bcc) \to (xayb, q_2, cc)$
$(xay, q_3, bzc) \to (xa, q_3, ybzc) \to (x, q_3, aybzc) \to (\epsilon, q_3, xaybzc) \to$
$(x, q_0, aybzc) \to (xx, q_1, ybzc) \to (xxy, q_1, bzc) \to (xxyy, q_2, zc) \to (xxyyz, q_2, c)$
$(xxyyz, q_3, z) \to (xxyy, q_3, zz) \to (xxy, q_3, yzz) \to (xx, q_3, yyzz) \to (xx, q_0, yyzz)$
$(xxy, q_4, yzz) \to \ldots$

Notes:

Notes:

Turing machines and word decision problems

- Turing machines with infinite tape are expressive enough to **recognize language of any type in Chomsky hierarchy**

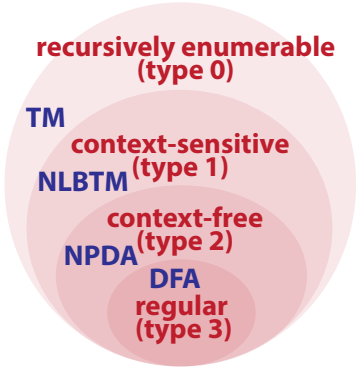
theorem

The set of languages that can be recognized by a (deterministic or non-deterministic) **Turing machine** is exactly the set of **recursively enumerable (type 0)** languages.

- is there an automaton model associated with **context-sensitive languages**?
- Turing machine that can never overwrite **blank symbol** on the tape is called **linearly bounded**

Kuroda's theorem

The set of languages that can be recognized by a **non-deterministic linearly bounded Turing machine** is exactly the set of **context-sensitive (type 1)** languages. → S-Y Kuroda, 1964



Venn diagram of Chomsky hierarchy and automata models

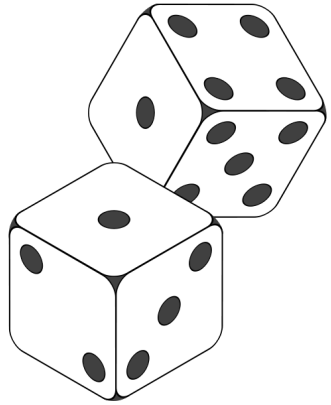
Deterministic vs. non-deterministic Turing Machines

- similar to finite automata, DTMs and NTMs are **equally expressive** w.r.t. word problems
- this implies that **for any NTM we can construct DTM that simulates it** (possibly requiring more time steps and more space on the tape)

idea to simulate NTM

1. use multiple virtual tapes, each capturing the state of the tape along multiple non-deterministic branches
2. since number of non-deterministic transitions is countable, we can fit those multiple (infinite) tapes in a single infinite tape of a DTM

- but: for some problems, **DTM may need exponentially more time** than NTM
- open question: $P = NP$ (generally believed to be false)



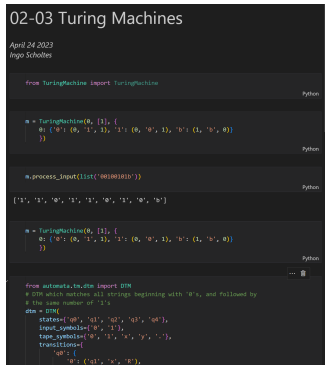
Notes:

Notes:

- We note that – similar to finite automata – the deterministic and non-deterministic variants of a Turing machine are equally powerful with respect to the recognition of languages. However, it is generally believed that non-deterministic Turing machines can solve some problems faster than deterministic Turing machine. This belief is famously expressed in the question whether $P = NP$, i.e. whether the set of problems that can be solved in a polynomial number of steps with a deterministic Turing machine (P) is equal to the set of problems that can be solved in a polynomial number of steps with a non-deterministic Turing machine (NP).
- For certain problems, a deterministic Turing machine may need exponentially more time than a non-deterministic Turing machine. Intuitively, there are problems where a given answer to a question can be validated in polynomial time. Such problems can often be solved by non-deterministic Turing machines in polynomial time, as we (conceptually) can try out multiple solutions in parallel via non-deterministic transitions and then halt as soon as we found a valid answer. If there is no known efficient constructive process for a solution, a deterministic Turing machine would instead need to try all possible solutions to the problem, which generally requires exponential time.

Practice Session

- we implement pushdown automata in python
- we implement Turing machines in python
- we use PDAs and Turing machines to accept languages



practice session

see notebook 04-01 and 04-02 in `gitlab` repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Turing machines and computability

- when is a function $f : \mathbb{N}^k \to \mathbb{N}$ "computable"?
- **intuitive notion of computability**, i.e. we can write a `python` program that computes $f(x)$ in finite time for any input x.
- concept of **computability can be formalized** based on Turing machines
- we say that $f : \mathbb{N}^k \to \mathbb{N}$ is **Turing-computable** iff there exists a Turing machine M such that for any $x_1, \ldots, x_k \in \mathbb{N}$ and $y \in \mathbb{N}$ we have $f(x_1, \ldots, x_k) = y$ iff $(\epsilon, q_0, x_1 \# \ldots \# x_k) \to^* (\square, q_e, y)$ for $q_e \in E$

Church-Turing Thesis

A function f is intuitively computable iff a Turing machine exists that computes f.

Alonzo Church, 1903 – 1995

image credit: Princeton University, fair use

Notes:

Notes:

Universal Turing Machines

- for given (word) problem, we can "construct" a Turing machine by defining state transitions, tape alphabet, etc.
- we can encode given Turing machine (incl. state transitions, tape, etc.) using finite sequence of symbols with finite alphabet (e.g. 0 and 1)
- we can define a **universal Turing machine** U that simulates any other Turing machine M encoded on the input tape of U

> "It is possible to invent a **single machine which can be used to compute any computable sequence**. If this machine U is supplied with a tape on the beginning of which is written the [standard description] of some computing machine M, then U will compute the same sequence as M." → AM Turing, 1936

- first theoretical concept of **universal computer** capable of solving any problem that can be solved in principle

character Neo from the movie "The Matrix"

→ Warner Bros. Entertainment Inc., fair use

Notes:

Stored Program computers

- modern computers are **universal**, i.e. we can build a single machine that can (in principle) solve any computable problem
- **stored-program computers** hold both instructions and input data in memory
- so-called **Von Neumann architecture** still basis for design of modern processors → J von Neumann, 1945
- concept related to universal Turing machine, i.e. processor can simulate any possible Turing machine (and thus execute any possible program)

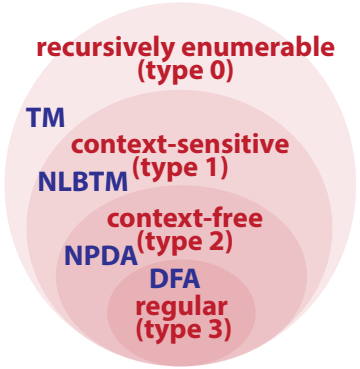
John von Neumann, 1903 – 1957

image credit: Los Alamos National Laboratory, fair use

Notes:

Decidable languages

- let L be a language and let M be a Turing machine
- we say that M solves word problem for L (i.e. it recognizes L) iff for all $w \in L$ it halts in a final state
- we say that M **decides language** L iff
  1. for all $w \in L$ it accepts w (i.e. it halts in final state) and
  2. for all $w \notin L$ it halts (and rejects w)
- can we use Turing machines to **decide** any formal language?
- are there languages that **can be recognized** but **cannot be decided** by a Turing machine?



Venn diagram of Chomsky hierarchy and automata models

The Halting problem

Halting problen

Consider language $L = \{M\#w | \text{Turing machine } M \text{ started with input } w \text{ halts } \}$. L is a type-0 language (i.e. recognizable by a Turing machine) but not decidable by a Turing machine.

proof idea

1. suppose there were a universal Turing machine H, which – for given Turing machine M and any input w – decides if M does **not** halt, i.e. $H(M, w)$ solves word problem for L^C
2. using H, we can define another universal Turing machine $T(M)$, which – for a given Turing machine M – does not halt if H decides that M halts with input M
3. what is the output of $H(T(M), M)$?

- Halting problem shows that some type-0 languages are **undecidable**
- in any sufficiently complex formal system we can formulate problems that are undecidable

Kurt Gödel, 1906 – 1978

image credit: Wikimedia Commons, public domain

Notes:

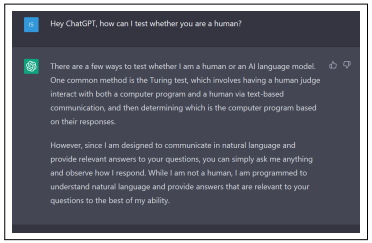
Notes:

- Note that the two statements (i) that the set of languages that can be recognized by a Turing machine is exactly the set of type 0 languages, and (ii) that the Halting problem is a type-0 language that **cannot be decided** by a Turing Machine do not contradict each other!
- Considering the definition from slide 06, to recognize a language L it is sufficient if the Turing machine halts after a finite amount of steps in a final state for each $w \in L$.
- To decide a language, it must additionally halt for any $w \notin L$.
- The Halting problem is an example for a *semi-decidable* type-0 language. Such languages are also called Turing-recognizable, Turing-acceptable or partially decidable.
- We also say that the word problem for any type-0 language is semi-decidable, i.e. while we can give a Turing machine that can decide (in finite time) whether $w \in L$, the machine may never terminate for $w \notin L$.

Turing test

- Turing machines can be used to formalize and explore concepts like “computation” and “algorithm”
- how can we **test whether a given algorithm or machine exhibits “intelligence”**, i.e. whether it can “think”?
- **no universally accepted definition of intelligence**
- Turing proposed the following **“imitation game”**

> I propose to consider the question, “Can machines think?”. This should begin with definitions of the meaning of the terms "machine" and "think." […] Instead of attempting such a definition I shall replace the question by another […] The new form of the problem can be described in terms of a game which we call the **'imitation game'**. […] What will happen when a machine takes the part of A in this game?" Will the interrogator decide wrongly as often when the game is played like this as he does when the game is played between a man and a woman? → A Turing, 1950

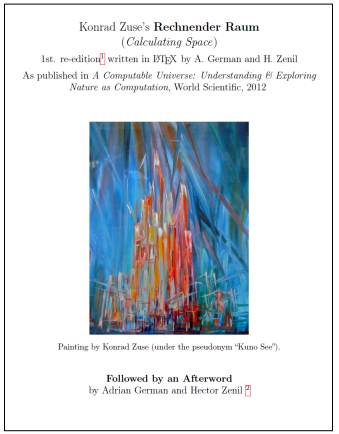


response from large language model **ChatGPT**

image credit: OpenAI screenshot, taken on 29/03/2023

In summary …

- we have covered **formal foundations of computing**
- formal languages and automata allow us to formalize **which problems are decidable**
- Turing machines provide **model of computation** that is believed to be **universal** → Church-Turing thesis
- relates to **deep philosophical questions**
  1. can we, in principle, algorithmically solve any problem that we can formalize? → no!
  2. are the laws of physics Turing-computable, i.e. **can the physical universe be viewed as Turing machine**? → ?
- basis to study concepts like **“artificial intelligence”** and answer question **whether machines can “think”**


Konrad Zuse's **Rechnender Raum**
(*Calculating Space*)
1st. re-edition written in LaTeX by A. German and H. Zenil
As published in *A Computable Universe: Understanding & Exploring Nature as Computation*, World Scientific, 2012

Painting by Konrad Zuse (under the pseudonym “Kuno See”).

Followed by an Afterword
by Adrian German and Hector Zenil

Notes:

Notes:

Self-study questions

1. Give an example for a grammar of a language that can be recognized by a Turing machine, but not by a PDA.
2. Define a Turing machine that accepts correctly bracketed arithmetic expressions.
3. Explain the difference between the question whether a Turing machine decides a language and whether it recognizes a language.
4. Explain why non-deterministic and deterministic Turing machines are equally expressive w.r.t. word problems.
5. Can the idea underlying the simulation of NTMs by DTMs be applied to pushdown automata? Explain your answer.
6. Give an example for a type-0 language that is not decidable by a Turing machine.
7. Explain the relationship between Universal Turing Machines and stored-program computers.
8. Test whether the Turing machine shown on slide 7 accepts the word *aaabbbccc*.
9. Define a Turing machine that given a binary representation of natural number x produces the binary representation of $x + 1$ as output.

References

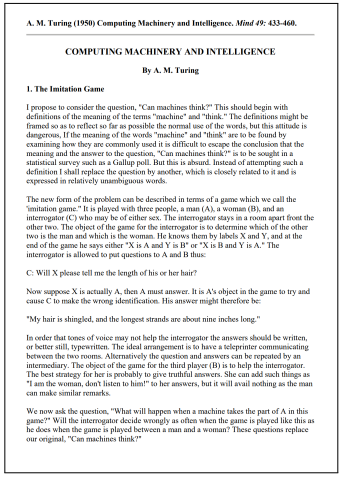
reading list

- AM Turing: **On Computable Numbers, with an Application to the Entscheidungsproblem**, 1936
- J von Neumann: **First Draft of a Report on the EDVAC**, 1945
- AM Turing: **Computing Machinery and Intelligence**, Mind 49, 433 – 460, 1950
- N Chomsky: **Three models for the description of language**, 1956
- S Scheinberg: **Note on the Boolean Properties of Context Free Languages**, Information and Control, 1960
- Y Bar-Hillel, M Perles, E Shamir: **On formal properties of simple phrase-structure grammars**, Zeitschrift für Phonetik, Sprachwissenschaft, und Kommunikationsforschung, 1961
- S-Y Kuroda: **Classes of languages and linear-bounded automata**, 1964
- K Zuse: **Rechnender Raum**, 1969
- D Knuth: **The Art of Computer Programming. Vol. 1: Fundamental Algorithms**, Addison–Wesley, 1973

acknowledgments

some examples are based on the following books and lectures:

- U Schöning: **Theoretische Informatik - kurzgefasst**, Spektrum Akadem. Verlag, 2001
- J Esparza: **Automata theory: An algorithmic approach**, lecture notes, 2020



A. M. Turing (1950) Computing Machinery and Intelligence. *Mind 49*: 433-460.

COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

1. The Imitation Game

I propose to consider the question, "Can machines think?" This should begin with definitions of the meaning of the terms "machine" and "think." The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous, If the meaning of the words "machine" and "think" are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, "Can machines think?" is to be sought in a statistical survey such as a Gallup poll. But this is absurd. Instead of attempting such a definition I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words.

The new form of the problem can be described in terms of a game which we call the 'imitation game." It is played with three people, a man (A), a woman (B), and an interrogator (C) who may be of either sex. The interrogator stays in a room apart front the other two. The object of the game for the interrogator is to determine which of the other two is the man and which is the woman. He knows them by labels X and Y, and at the end of the game he says either "X is A and Y is B" or "X is B and Y is A." The interrogator is allowed to put questions to A and B thus:

C: Will X please tell me the length of his or her hair?

Now suppose X is actually A, then A must answer. It is A's object in the game to try and cause C to make the wrong identification. His answer might therefore be:

"My hair is shingled, and the longest strands are about nine inches long."

In order that tones of voice may not help the interrogator the answers should be written, or better still, typewritten. The ideal arrangement is to have a teleprinter communicating between the two rooms. Alternatively the question and answers can be repeated by an intermediary. The object of the game for the third player (B) is to help the interrogator. The best strategy for her is probably to give truthful answers. She can add such things as "I am the woman, don't listen to him!" to her answers, but it will avail nothing as the man can make similar remarks.

We now ask the question, "What will happen when a machine takes the part of A in this game?" Will the interrogator decide wrongly as often when the game is played like this as he does when the game is played between a man and a woman? These questions replace our original, "Can machines think?"

Notes:

Notes: